LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior full-stack engineer building **FinTech** and real-time data platforms with **Python 3.10–3.12**, **FastAPI 0.95–0.110**, **React 17–18**, and **Angular 12–18**, delivering low-latency APIs, async pipelines, and high-concurrency dashboards. Strong in **secure REST** design (OAuth2/OIDC, JWT, RBAC), production hardening (rate limits, idempotency, retries), and data performance on **PostgreSQL 12–16** with indexing and query-plan tuning. Experienced with **Docker** and **Kubernetes** deployments on **AWS**, plus automated testing, CI checks, and clean modular architecture to keep releases predictable under pressure.

SKILLS

- **Languages**: **Python 3.9–3.12**, JavaScript (ES2019–ES2024), TypeScript 4.8–5.5, SQL
- **Backend & APIs**: **FastAPI 0.95–0.110**, asyncio, REST (OpenAPI 3.0/3.1), WebSockets/SSE, background workers, schema validation, auth (**OAuth2/OIDC**, **JWT**, **RBAC**)
- **Frontend**: **React 17–18** (hooks, performance profiling), **Angular 12–18** (RxJS, state patterns), dashboards, responsive UI architecture
- **Data**: **PostgreSQL 12–16** (indexes, migrations, EXPLAIN), Redis 6–7 (caching), event-driven query patterns
- **Cloud & DevOps**: **AWS** (API Gateway, Lambda, ECS/EKS, S3, RDS, CloudWatch, IAM), **Docker**, **Kubernetes**, Helm, CI/CD with GitHub Actions
- **Testing & Quality**: Pytest, Jest, React Testing Library, Cypress/Playwright, contract testing, load testing, linting/formatting gates
- **Tooling**: **Git**, trunk-based workflows, code reviews, feature flags, incident runbooks

WORK HISTORY

Senior Software Engineer
August 2021 to November 2025

RavenTrack

- Built low-latency **FastAPI** services with async I/O and backpressure controls, keeping P95 latency stable while traffic and concurrency increased.
- Designed **secure REST** endpoints with OAuth2/OIDC + JWT and strict request validation, preventing auth bypass edge cases and tightening auditability.
- Fixed a high-impact bug where concurrent retries created duplicate transactions, resolving it with **idempotency keys**, unique constraints, and replay-safe handlers.
- Implemented real-time streaming updates to dashboards using **WebSockets/SSE**, adding reconnect logic, versioned payloads, and graceful degradation for outages.
- Optimized **PostgreSQL** performance by rewriting slow joins, adding covering indexes, and using query plans to remove sequential scans under peak load.
- Reduced UI rendering stalls by restructuring state updates and memoization in **React 18**, keeping dashboards responsive during high-frequency market events.
- Modernized an aging UI module by migrating legacy patterns into a modular **Angular 18** feature shell, improving maintainability and lowering regression risk.
- Containerized services with **Docker** and standardized runtime configs, eliminating “works on my machine” issues across local, CI, and staging.
- Operated **Kubernetes** workloads with health probes, resource limits, and autoscaling rules, preventing cascading failures during sudden traffic spikes.
- Resolved production incidents caused by misconfigured timeouts between services, tuning client/server budgets and adding retries with jittered backoff.
- Introduced contract tests for API schemas, catching breaking changes early and preventing frontend integration failures during rapid release cycles.
- Migrated a legacy synchronous workflow to an async queue-driven model, adding retries, DLQ handling, and observability for stuck-job recovery.
- Implemented feature flags for risky launches, enabling safe rollouts, fast rollback, and measurable impact tracking without full redeploys.
- Hardened security posture by adding rate limiting, RBAC checks, and structured audit logs, supporting compliance expectations in finance workflows.
- Mentored engineers on clean architecture boundaries and testing strategy, improving code review quality and reducing post-release defect rates.

Software Engineer
June 2018 to July 2021

Infinite Loop

- Built backend services in **Python** with clean OOP boundaries, adding structured validation and consistent error contracts so clients behaved predictably.
- Implemented high-concurrency API endpoints in **FastAPI**, tuning async execution paths and pooling to keep throughput high under load tests.

- Fixed a recurring production bug where stale cache keys served incorrect balances, solving it with versioned keys, TTL policy, and invalidation hooks.
- Created data-driven dashboards in **React 16** with resilient polling/streaming fallbacks, keeping UX stable when upstream feeds degraded.
- Improved state management performance by reducing unnecessary re-renders and batching updates, preventing UI freezes during bursty event streams.
- Migrated database migrations from ad-hoc scripts into repeatable pipelines, adding safety checks to avoid locks and long-running transactions.
- Added **PostgreSQL** indexes and query rewrites to eliminate timeouts on reporting endpoints, using EXPLAIN to verify plan improvements.
- Introduced containerized dev environments with **Docker**, reducing onboarding time and keeping dependency drift from breaking builds.
- Integrated core services with **AWS** primitives (IAM policies, logs/metrics), improving operational visibility and access control discipline.
- Expanded test coverage with Pytest + Jest, including edge-case fixtures for race conditions, preventing reintroductions of concurrency bugs.
- Collaborated closely with product to ship new fintech features safely, using feature flags and staged rollouts to protect critical flows.

Software Developer
October 2015 to May 2018
Datamart

- Built server-side modules in **Python** and SQL focused on data correctness, ensuring reporting outputs matched business rules under evolving requirements.
- Fixed long-standing query issues by refactoring joins and adding indexes, improving report generation time and reducing database load spikes.
- Implemented robust ETL-style pipelines with retry logic and validation, preventing bad upstream data from silently corrupting downstream tables.
- Supported legacy-to-modern transitions by wrapping older endpoints with versioned APIs, keeping existing clients compatible during upgrades.
- Resolved a critical bug where timezone handling skewed daily aggregates, correcting it with UTC normalization and backfilled repair scripts.
- Improved developer workflow with **Git** conventions and review checklists, reducing merge conflicts and stabilizing releases across the team.
- Added automated tests for core calculations and edge cases, catching regressions before deployments and improving stakeholder trust in numbers.
- Introduced containerized local setups using **Docker** to standardize dependencies, preventing environment drift from blocking delivery.
- Partnered with analysts to implement new data features quickly, translating ambiguous requirements into validated schemas and reliable outputs.

EDUCATION

Bachelor's degree in Computer Science
2011 to 2015
University of Amsterdam

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

Transaction API + Simple Dashboard

- Built a small **FastAPI** service in **Python** that provides **REST** endpoints for creating and listing transactions with **JWT** security and clear validation.
- Created a basic **React 18** page that shows the latest transactions and status, then fixed a lag issue by reducing unnecessary re-renders and keeping state updates minimal.
- Improved **PostgreSQL** speed by adding an index and using pagination after a slow query caused occasional timeouts under concurrent requests.